

Tìm kiếm ảnh ký tự Hán-Nôm tương đồng

Từ histogram tới embedding, và fine-tune bằng cặp dương sinh từ font

Đồ án môn học · mã nguồn, dữ liệu và mô hình kèm theo · xem Phụ lục A để tái lập

Tóm tắt

Báo cáo trình bày một hệ thống tìm kiếm ảnh ký tự Hán-Nôm tương đồng trên corpus 26.044 glyph, và quá trình cải thiện nó qua ba bước đo được. **Bước một:** thay bộ trích đặc trưng ResNet18 của bản gốc bằng tháp ảnh Chinese-CLIP, nâng `radical@k` từ 0,2316 lên 0,5361 (2,31 lần). **Bước hai:** xây dựng phép đánh giá trên ảnh scan thật thay vì chỉ dựa vào chỉ số proxy — 59 ảnh viết tay, nhãn giải mã từ tên file rồi được người đọc kiểm lại; trên tập này mô hình tốt nhất chỉ đạt `hit@1` = 0,0847, thấp hơn hẳn mức mà chỉ số proxy gợi ý. **Bước ba:** fine-tune bằng học tương phản có giám sát (SupCon) trên 141.981 ảnh render đa font, nhãn là mã Unicode, đưa `hit@1` trên chính 59 ảnh scan đó lên **0,5932** — hai khoảng tin cậy 95% rời hẳn nhau. Trên bài toán phụ (xếp hạng trong nhóm ký tự cùng âm Quốc Ngữ), độ chính xác top-1 tăng từ 0,4746 lên **0,8136**. Các lớp Unicode bị giữ lại hoàn toàn ngoài tập huấn luyện đạt kết quả bằng đúng các lớp đã huấn luyện (0,9939 so với 0,9954), nên phần cải thiện không đến từ việc ghi nhớ. Báo cáo cũng ghi lại đầy đủ các kết quả âm — trong đó có một lần huấn luyện bị sụp đổ biểu diễn (representation collapse) — vì nguyên nhân của chúng có tính lặp lại.

1. Bài toán

Cho một ảnh glyph Hán-Nôm, tìm những ký tự trong corpus có hình dạng gần nó nhất. Bài toán được chia làm hai phần, tương ứng hai tình huống sử dụng khác nhau:

- **Phần 1 — tìm trên toàn corpus.** Không biết trước ký tự là gì; xếp hạng toàn bộ 26.044 ứng viên. Đây là tình huống khó, dùng khi số hoá một trang chữ chưa được phiên âm.
- **Phần 2 — xếp hạng trong nhóm cùng âm Quốc Ngữ.** Đã biết chữ đó đọc là gì (ví dụ "trăm"), cần chọn đúng tự dạng trong số các ứng viên cùng âm — trung vị 7 ứng viên. Đây là tình huống thường gặp khi hiệu đính một bản phiên âm đã có.

Đóng góp của nhóm, theo thứ tự giá trị:

1. Một **phép đánh giá trên dữ liệu thật** thay cho chỉ số proxy: 59 ảnh scan viết tay có nhãn, kèm thang tin cậy do người gán đọc lập với mọi điểm số của mô hình (mục 4).
2. **Fine-tune** nâng `hit@1` trên scan thật từ 0,0847 lên 0,5932, kèm phép kiểm chứng minh kết quả không phải do ghi nhớ (mục 5.4, 5.5).
3. Một **quy trình sinh dữ liệu huấn luyện** từ corpus vốn chỉ có một ảnh mỗi ký tự, bằng render đa font cộng mô hình suy giảm ảnh được hiệu chỉnh theo số đo của scan thật (mục 3.4).
4. **Bốn kết quả âm** được ghi lại đầy đủ, gồm phân tích trần của chỉ số `radical@k` và một lần sụp đổ biểu diễn khi huấn luyện (mục 6).

2. Dữ liệu

Bảng 1. Các tập dữ liệu. Ba tập đầu là đầu vào cho trước; hai tập sau do nhóm tạo hoặc gán nhãn.

Tập	Kích thước	Vai trò
images/	26.044 ảnh JPG 70×70	Corpus tra cứu. Mỗi ký tự đúng một ảnh, tên file là mã Unicode.
final_characteristics-v2.xlsx	31.208 dòng	Bộ thủ, số nét, phân rã IDS. 5.164 dòng không có ảnh nên bị loại.
QuocNgu_SinoNom_Dic.xlsx	—	Ảnh xạ âm Quốc Ngữ ↔ ký tự Sino-Nôm, dùng cho Phần 2.

test_images/	59 ảnh scan	Tập đánh giá thật. Chữ viết tay, kích thước không đồng nhất (88×96 ... 178×162).
output/rendered/	131.604 ảnh	Sinh ra để huấn luyện: mỗi ký tự render qua tối đa 7 font.

Đặc điểm quyết định toàn bộ thiết kế: **corpus chỉ có một ảnh cho mỗi ký tự**. Học metric cần cặp dương — hai ảnh khác nhau của cùng một lớp — nên trong dữ liệu gốc *không tồn tại cặp nào*. Mục 3.4 giải quyết đúng chỗ này.

Ảnh trong test_images/ được đặt tên bằng kiểu gõ Telex (cofn.jpg = "còn", nguowosi.jpg = "người"), nên giải mã ngược ra chữ Quốc Ngữ rồi tra từ điển sẽ được **tập ký tự đúng** cho ảnh đó. Đây là nhân thật, không phải proxy — và là lý do tập 59 ảnh này có giá trị lớn hơn nhiều so với kích thước của nó.

3. Phương pháp

3.1. Trích đặc trưng và tìm kiếm

Mọi backend đưa ảnh về một vector đã chuẩn hoá L2, nên tích vô hướng chính là cosine. Tìm kiếm dùng Faiss IndexFlatIP — tìm chính xác, không xấp xỉ (lý do ở mục 5.3). Kết quả trích đặc trưng được cache ra .npz nên chỉ trả chi phí một lần.

Bản gốc dùng ResNet18 pretrain ImageNet, thay lớp conv1 bằng một lớp 1 kênh khởi tạo ngẫu nhiên. Nhóm giữ nguyên bản này **nguyên vẹn từng bit** làm mốc "trước", và thêm năm backend để so: một biến thể resnet18-gray sửa đúng một dòng, ba mô hình Chinese-CLIP ba cỡ, và DINOv2.

3.2. Phần 2 — xếp hạng trong nhóm cùng âm

Với một âm Quốc Ngữ, từ điển cho ra tập ứng viên; hệ thống xếp hạng chúng theo độ giống ảnh query. Hai cách chấm điểm được so sánh: **histogram** (đặc trưng thủ công của bản gốc) và **embedding** (dùng lại vector của Phần 1). Trên phép thử tự truy hồi, embedding đạt 64/64 còn histogram 62/64; đáng chú ý hơn, histogram nén cả nhóm ứng viên vào một dải điểm rộng 0,003, tức nó gần như không phân biệt được gì bên trong nhóm.

3.3. Sinh cặp dương bằng render đa font

Vì mỗi ký tự chỉ có một ảnh, nhóm render lại từng mã Unicode qua bảy font Hán-Nôm khác nhau. Cùng một lớp, khác kiểu chữ — đúng loại cặp dương cần để dạy mô hình bỏ qua khác biệt tự dạng. Độ phủ được kiểm bằng bảng cmap của từng font chứ không phỏng đoán: hanamin 99,9%, babelstone 88,3%, iming 83,7%, lxgw-kai 71,3%, noto-serif và noto-sans 59,9%, zhuque-fangsong 42,3%; hợp nhất lại phủ 26.033/26.044 ký tự.

Ký tự không có trong cmap bị bỏ qua, không render. Nếu render bừa, font trả về ô tofu — một ô vuông rỗng giống hệt nhau ở mọi ký tự — đưa vào huấn luyện sẽ đầu độc cặp dương.

Nhưng tập render sạch **quá dễ**: ảnh render tự truy hồi đúng chữ ở hit@1 = 0,7018 trong khi scan thật chỉ đạt 0,0847, chênh gần chín lần. Nghĩa là phần lớn khoảng cách tới thực tế **không nằm ở font**. Nhóm đo trực tiếp sáu chỉ số ảnh trên hai tập để tìm chỗ lệch:

Bảng 2. Khoảng cách đo được giữa scan thật và ảnh render. Hàm suy giảm mô phỏng đúng sáu trục này.

Chỉ số	Scan thật	Render	Tỉ lệ	Ý nghĩa
edge	15,6	102,3	0,15×	scan mờ hơn nhiều — khoảng cách lớn

				nhất
contrast	91,6	232,8	0,39×	mức xám nhạt, không đen tuyền
thick	0,051	0,028	1,82×	nét scan dày gần gấp đôi
fill	0,349	0,206	1,69×	hệ quả của nét dày
noise	10,1	2,7	3,77×	hạt nhiễu giấy và cảm biến
bg	254,0	255,0	≈1×	nền giấy hơi ngà

Hàm `degrade()` mô phỏng sáu trục đo theo thứ tự vật lý của một trang scan: tay viết → mực loang trên giấy → mất phân giải khi chụp → nhiễu quang học → giấy ngà và tương phản kém → nhiễu cảm biến. Mọi phép biến đổi đều là nhiễu ảnh, **không đụng tới cấu trúc bộ thủ**, nên nhân vẫn đúng. Kết quả: hit@1 của tập render tụt từ 0,7018 xuống 0,1986 — chỉ còn dễ hơn scan thật 2,8 lần thay vì 9 lần. Phần 2,8 lần còn lại là khác biệt **cấu trúc** (bút lông, lối hành thư: nét nối liền, tỉ lệ bộ phận xê dịch, có nét bị lược) và không thể sinh ra từ một ảnh in bằng xử lý ảnh.

3.4. Fine-tune

Mục tiêu học là **bất biến kiểu chữ**: cùng một mã Unicode qua font khác nhau phải cho cùng một vector. Nhân duy nhất là mã Unicode — cố ý **không** dùng RADICAL hay STROKE_NUM, vì huấn luyện trên một nhân rồi báo cáo chính chỉ số của nhân đó là đo lại tập huấn luyện chữ không phải đánh giá.

Bảng 3. Cấu hình fine-tune.

Thành phần	Lựa chọn
Backbone	Chinese-CLIP ViT-B/16, giữ nguyên <code>visual_projection</code> (512 chiều)
Hàm mất mát	SupCon (InfoNCE có giám sát), nhiệt độ 0,07
Lấy mẫu	P×K: 24 lớp × 2 view mỗi batch
Dữ liệu	141.981 ảnh / 23.440 lớp (đã trừ held-out)
Suy giảm ảnh	<code>degrade()</code> chạy trực tuyến trong DataLoader, $p = 0,6$, $strength = 1,0$
Tối ưu	AdamW, lr $1e-5$, warmup 300 bước, cosine, bf16, gradient checkpointing
Chi phí	3,7 GB VRAM , ~4 phút/epoch, 6 epoch (~30 phút trên một H100 dùng chung)

Hai lựa chọn trong bảng trên là **điều kiện sống còn chứ không phải tinh chỉnh**, và cả hai đều đến từ một lần thất bại được ghi ở mục 6.3:

- **Lấy mẫu P×K.** Một batch 48 ảnh rút ngẫu nhiên từ 23.440 lớp chỉ chứa khoảng 0,05 cặp dương theo kỳ vọng — gần như mọi batch sẽ không có gì để kéo lại gần nhau, và hàm mất mát trở thành hằng số. P×K bảo đảm mỗi ảnh luôn có đúng K−1 ảnh cùng lớp.
- **SupCon thay vì softmax có biên góc (ArcFace).** SupCon không có tham số học được nào trong hàm mất mát, nên không có một head khởi tạo ngẫu nhiên bơm nhiễu ngược về backbone.

Suy giảm ảnh chạy **trực tuyến lúc nạp dữ liệu** chứ không render sẵn ra đĩa: bộ sinh số ngẫu nhiên gieo theo bộ ba (seed, epoch, chỉ số ảnh) nên mỗi epoch mỗi ảnh có một biến thể khác, vẫn tái lập được, và không tốn thêm dung lượng nào.

4. Phương pháp đánh giá

Phần lớn công sức của đồ án nằm ở chỗ này, vì một chỉ số sai làm mọi kết luận phía sau vô nghĩa. Ba tầng đánh giá, xếp theo độ tin cậy tăng dần:

- **Chỉ số proxy** trên toàn corpus: `radical@k` (tỉ lệ lảng giềng cùng bộ thủ), `stroke_mae@k`, `ids_jaccard@k`. Rẻ, chạy được trên 26.044 ký tự, nhưng là **nhân yếu** — mục 6.2 cho thấy chúng có trần riêng.
- **Nhân thật từ tên file** trên 59 ảnh scan: giải mã Telex ra chữ Quốc Ngữ, tra từ điển ra tập ký tự đúng. `hit@k` = tỉ lệ ảnh có ít nhất một ký tự đúng trong top-k. Mức ngẫu nhiên khoảng $4,7 \cdot 10^{-4}$, nên đây là bài kiểm tra khó và trung thực.
- **Nhân do người gán** cho Phần 2: với mỗi ảnh, người đọc chọn đúng một ứng viên trong nhóm cùng âm. Đã gán đủ 59/59 ảnh.

Cột confidence trong bảng nhân là **đánh giá của người đọc ảnh, không suy ra từ bất kỳ điểm số nào của mô hình**. Điều này là cố ý: phép kiểm độ nhạy dùng chính cột này để lọc ra tập nhân chắc chắn rồi chấm lại mô hình trên đó, nên nếu confidence lấy từ điểm similarity thì phép kiểm ấy thành vòng tròn — lấy mô hình chấm mô hình. Thang gồm ba mức: **high** (nét quyết định đọc được, mọi ứng viên còn lại khác ở bộ thủ hoặc số nét thấy rõ), **med** (còn ít nhất một biến thể gần trùng chỉ khác ở chi tiết đã mờ trong bản scan), **low** (nét quyết định không đọc được, lựa chọn dựa vào văn cảnh nhiều hơn hình). Phân bố cuối cùng là 53 high / 4 med / 2 low.

Các ảnh scan là những dòng mở đầu *Truyện Kiều*, nên nhân được gán bằng so hình trước rồi đối chiếu với chính tả Nôm chuẩn của tác phẩm. **Khi hình và văn cảnh mâu thuẫn thì hình thắng**, và confidence bị hạ xuống med — bản scan viết "qua" là 𠂔 chứ không phải 過, "năm" là 𠂔 chứ không phải 年.

Mọi tỉ lệ đều kèm khoảng tin cậy nhị thức 95%. Với $n = 59$ và $p \approx 0,5$, khoảng tin cậy rộng khoảng ± 13 điểm, nên bất kỳ cải thiện nào dưới ~ 15 điểm đều không chứng minh được. Đây là giới hạn cứng của tập test hiện có, và báo cáo tuân thủ nó một cách nhất quán: trong toàn bộ đồ án chỉ có đúng một so sánh vượt qua được ngưỡng này (mục 5.4).

5. Kết quả

5.1. So sánh backend trên toàn corpus

Bảng 4. Toàn bộ corpus 26.044 glyph, top-K = 20. Đậm là tốt nhất mỗi cột.

Backend	dtype	chiều	radical@k ↑	stroke_mae@k ↓	ids_jaccard@k ↑	img/s (H100)
resnet18 (bản gốc)	fp32	512	0,2316	2,8942	0,1031	3950
resnet18-gray	fp32	512	0,2202	2,5658	0,1020	3937
chinese-clip (B/16)	fp32	512	0,4843	2,6296	0,1997	358
chinese-clip-large	fp16	768	0,5361	2,6992	0,2113	354
chinese-clip-huge	fp16	1024	0,5147	2,6547	0,2144	283
dinov2	fp32	1536	0,3081	2,3397	0,1251	306

Chinese-CLIP thắng với biên độ lớn: `radical@k` gấp **2,31 lần** bản gốc và `ids_jaccard` gấp 2,05 lần, mà gần như không tốn thêm thời gian (354 so với 358 ảnh/giây). Giả thuyết ban đầu — một prior quen với glyph Hán sẽ thắng một CNN ImageNet — được xác nhận.

DINOv2 thua ở bộ thủ nhưng **thắng ở số nét**, và chỉ trùng 12,9% kết quả top-k với họ CLIP. Nó không đơn thuần là kém hơn mà mạnh ở một trục khác, nên là ứng viên tốt cho ensemble.

5.2. Trần của chính chỉ số proxy

Một câu hỏi được đặt ra: $\text{radical}@k \approx 0,5$ có phải đã bão hoà? Nhóm kiểm hai giả thuyết.

Giả thuyết "tập top-k thiếu ứng viên đúng" bị bác bỏ: trần lý thuyết của $\text{radical}@20$ là 0,9862, tức chỉ 3% query bị chạm trần; và $\text{radical}@1$ cũng chỉ đạt 0,6331, nên chỗ nghẽn nằm ngay ở lảng giềng gần nhất chứ không ở đuôi danh sách.

Nhưng phép kiểm làm lộ ra một điều khác, và quan trọng hơn: **RADICAL không phải nhãn thị giác thuần túy**. Bộ thủ chỉ thực sự xuất hiện trong hình chữ ở 56,7% số ký tự — phần còn lại mang bộ thủ vì lý do từ nguyên hoặc quy ước tra cứu, không phải vì trông giống. Một oracle *biết trước* phân rã IDS cũng chỉ đạt $\text{radical}@20 = 0,7338$. So với trần thực tế đó, *chinese-clip-large* (0,5361) đang ở **73% quãng đường** và **24,4 lần** mức ngẫu nhiên. Kết luận: chưa bão hoà, nhưng cũng không nên tiếp tục tối ưu cho một chỉ số mà bản thân nó chỉ đúng ở hai phần ba.

5.3. Có nên thay tìm kiếm chính xác bằng chỉ mục xấp xỉ?

Đã đo trên dữ liệu thật: HNSW đạt recall 99,1% so với tìm chính xác ở $N = 26.044$. Nhưng ở cỡ corpus hiện tại, **tìm kiếm chỉ chiếm 6% tổng thời gian** — 94% còn lại là trích đặc trưng, thứ mà ANN không đụng tới. Điểm hoà vốn nằm quanh $N \approx 300.000$. Kết luận: giữ IndexFlatIP. Nguyên lý về ANN là đúng, nhưng chưa tới lúc dùng.

5.4. Kết quả trên ảnh scan thật — phép đo chính

Đây là phép đo duy nhất không phải proxy, và là con số nên dùng để đánh giá đồ án. Không một pixel nào của `test_images/` đi vào huấn luyện; điều này được bảo đảm bằng một `assert` chạy trước mỗi lần huấn luyện chứ không bằng quy ước.

Bảng 5. Phần 1 — tìm trên toàn bộ 26.044 ký tự, $n = 59$ ảnh scan thật.

Mô hình	hit@1	hit@10	hit@20	MRR	KTC 95% của hit@1
chinese-clip zero-shot (B/16)	0,0169	0,0678	—	0,0327	—
chinese-clip-large zero-shot	0,0847	0,1864	0,2034	0,1143	[0,028; 0,187]
chinese-clip-ft (fine-tune)	0,5932	0,7966	0,8644	0,6691	[0,457; 0,719]

Hai khoảng tin cậy rời hẳn nhau. Đây là so sánh duy nhất trong đồ án mà $n = 59$ đỡ nổi một kết luận — và không phải vì mẫu lớn hơn, mà vì hiệu ứng đủ lớn: 35/59 so với 5/59, hơn mức ngẫu nhiên 1.250 lần.

Trước fine-tune, kết quả thấp **không phải vì mô hình kém mà vì lệch phân phối**. Triệu chứng là hiệu ứng hub: ký tự 攢 được trả về ở vị trí một cho sáu query khác nhau. Query là chữ viết tay lỗi hành thư, corpus là khai thư in bằng font; hai phân phối này khác nhau ở cả nét lẫn hình học. Nhóm đã thử chuẩn hoá hình học (cắt sát nét mực, đệm vào khung vuông) và kết quả nằm trong khoảng nhiễu — nghĩa là phần lệch hình học có thật nhưng không phải nguyên nhân chính. Fine-tune mới là thứ giải quyết được.

Phần 2, trên cùng bộ nhãn do người gán:

Bảng 6. Phần 2 — xếp hạng trong nhóm cùng âm Quốc Ngữ, $n = 59$.

Phương pháp	Đúng top-1	MRR	KTC 95%
Chọn ngẫu nhiên trong nhóm	0,1987	—	—
histogram (bản gốc)	26/59 = 0,4407	0,6070	[0,312; 0,576]
embedding zero-shot (large)	28/59 = 0,4746	0,6566	[0,343; 0,609]
embedding fine-tune	48/59 = 0,8136	0,8983	[0,691; 0,903]

Phép thử độ nhảy trên 53 nhãn high cho $45/53 = 0,8491$ so với histogram 0,4717, nên kết luận không phụ thuộc vào sáu nhãn med/low còn lại.

Bảng 6 **sửa lại một kết luận trước đó của chính nhóm**. Với mô hình zero-shot, khoảng cách giữa embedding và histogram chỉ là *hai ảnh trên 59* — nằm gọn trong nhiều — nên khuyến nghị dùng embedding khi ấy phải dựa vào chất lượng điểm số chứ không dám viện độ chính xác. Sau fine-tune, khoảng cách là 22 ảnh và hai khoảng tin cậy rời nhau: độ chính xác đã đủ để tự đứng làm lý do.

5.5. Có phải mô hình chỉ ghi nhớ 23.440 danh tính?

Đây là phản biện hiển nhiên nhất với một kết quả tăng mạnh như vậy, nên nhóm thiết kế phép kiểm cho nó ngay từ đầu. **2.604 mã Unicode bị gỡ toàn bộ view khỏi tập huấn luyện** — chia theo *lớp* chứ không theo ảnh, vì chia theo ảnh thì lớp nào cũng có mặt ở cả hai bên và phép đo mất sạch ý nghĩa. Query là một bản render font, gallery là ảnh corpus.

Bảng 7. $\text{hit}@1$ render $\rightarrow$ corpus. Hai mẫu cùng cỡ 2.604 lớp và cùng cách chọn ngẫu nhiên.

	Lớp đã thấy	Lớp chưa thấy
Trước huấn luyện	0,7938	0,8041
Sau huấn luyện	0,9954	0,9939

Bằng nhau. Không có khoảng cách ghi nhớ. Điều này khớp với cơ chế: SupCon không có tâm lớp học được nên không có gì để nhớ — thứ nó học là *phép so hai ảnh*, và phép đó chuyển sang ký tự chưa từng gặp nguyên vẹn. Cần nói thêm rằng hai mẫu phải được chọn ngẫu nhiên cùng cách: nếu lấy "lớp đã thấy" theo thứ tự mã Unicode thì mẫu sẽ thiên về khối CJK cơ bản (chữ phổ thông, tự dạng quen) trong khi held-out rải đều cả Ext-B, và hai con số không còn so được với nhau.

6. Kết quả âm

Bốn kết quả dưới đây đều là thất bại hoặc bác bỏ, và đều được giữ lại vì chúng loại trừ được những hướng mà người đọc báo cáo này rất dễ thử lại.

6.1. Giả thuyết "lỗi nằm ở conv1" — sai

Bản gốc thay conv1 bằng lớp 1 kênh khởi tạo ngẫu nhiên, tức vứt bỏ bộ lọc tầng đầu đã pretrain. Giả thuyết tự nhiên là chỉ cần vá dòng đó. Nhóm dựng resnet18-gray gieo conv1 bằng tổng các bộ lọc RGB pretrain — kết quả radical@k **giảm nhẹ** (0,2202 so với 0,2316). Sửa một dòng không cứu được kiến trúc; đây chính là bằng chứng biện minh cho việc đổi hẳn sang ViT thay vì vá baseline.

6.2. Scale có lợi, rồi đảo chiều

base $\rightarrow$ large là +10,7% tương đối trên radical@k, nhưng large $\rightarrow$ huge là **-4,0%** trong khi tốn thêm 33% số chiều và 20% thời gian. Đường cong đã quay đầu. Đây là chữ ký của "thêm dung lượng cho một

mục tiêu huấn luyện sai domain" — và cũng chính là lý do khiến fine-tune trở thành hướng đáng làm nhất, thay vì đổi sang checkpoint lớn hơn.

6.3. ArcFace gây sụp đổ biểu diễn

Thiết kế fine-tune đầu tiên dùng ArcFace trên 23.440 lớp (scale 32, biên 0,30). Sau một epoch:

Bảng 8. Diễn biến khi huấn luyện bằng ArcFace.

Chỉ số	Trước	Sau 1 epoch
render → corpus, lớp đã thấy	0,7938	0,0035
render → corpus, lớp chưa thấy	0,8041	0,0050
scan thật hit@1	0,0169	0,0000
hàm mất mát	20,5	19,4 (gần như đứng yên)

Sụp ở **cả hai** phía nên đây không phải overfit mà là **sụp đổ biểu diễn**: mọi ảnh dồn về một vector. Nguyên nhân nằm ở hình dạng dữ liệu chứ không phải siêu tham số — 23.440 lớp mà mỗi lớp chỉ khoảng 6 ảnh. Head ArcFace khởi tạo ngẫu nhiên với 23.440 tâm không bao giờ kịp tổ chức, nên thứ nó truyền ngược về backbone gần như là nhiễu. Và vì Adam chuẩn hoá theo độ lớn gradient, "tốc độ học nhỏ" $1e-5$ vẫn dịch chuyển trọng số đủ để phá cấu trúc pretrain trong khoảng 3.000 bước; cắt chuẩn gradient không cứu được, vì nhiễu bị cắt chuẩn vẫn là nhiễu. Đây là lý do mục 3.4 chọn SupCon.

Ba công cụ chẩn đoán được thêm vào sau lần này, và nên có trong bất kỳ thí nghiệm tương tự nào: (a) đánh giá **giữa epoch** để sự cố lộ ra sau một phút thay vì sau một giờ; (b) một cột đo **cosine trung bình giữa 2.000 vector corpus ngẫu nhiên** — nó chạy về 1,0 khi sụp đổ, và nếu không có nó thì "học hỏng" với "sụp đổ" trông giống hệt nhau trên hit@1 trong khi hai thứ đó phải sửa theo hai cách khác nhau; (c) biết trước chữ ký sụp đổ của SupCon là hàm mất mát đứng đúng ở $\ln(P \cdot K - 1)$. Trong lần chạy thành công, cột cosine đi từ **0,890 xuống 0,000** — tức không gian giãn ra tới mức gần trực giao. Con số 0,890 lúc zero-shot tự nó giải thích vì sao khả năng phân biệt trước đây yếu.

6.4. Một nhãn sai đã tự làm giảm kết quả của chính nhóm

Khi soát lại 11 dòng med/low, nhóm phát hiện một nhãn gán sai: bộ bên trái của chữ là $\pm$ (nét dọc dừng ở vạch ngang) chứ không phải $\mp$ (móc kéo xuống dưới vạch). Sửa lại làm một ảnh chuyển từ cột embedding sang cột histogram, khiến khoảng cách giữa hai phương pháp **co từ bốn ảnh xuống hai** — tức đi ngược lợi ích của kết luận mà nhóm đang muốn chứng minh. Ghi lại điều này vì nó là thước đo trực tiếp cho việc một bảng số ở $n = 59$ nhạy tới mức nào với đúng một nhãn.

7. Hạn chế

- **$n = 59$ là toàn bộ dữ liệu scan hiện có**, và là nút thắt lớn nhất. Mọi so sánh trong đồ án trừ mục 5.4 đều nằm trong khoảng nhiễu. Cần $n \approx 150$ để khoảng tin cậy xuống ± 8 điểm.
- **Mô hình fine-tune chỉ được đo trên một lần chạy, một seed**. Chênh lệch giữa các epoch (0,51–0,59) đã bằng cỡ nhiễu của phép đo, nên 0,5932 nên đọc là "quanh 0,55".
- **Backbone fine-tune là ViT-B/16 chứ không phải large**, vì large không vừa VRAM lúc huấn luyện trên máy dùng chung. Chưa biết fine-tune large sẽ cho kết quả bao nhiêu.
- Nhãn Phần 2 do **so hình** mà ra, không phải do chuyên gia Hán-Nôm đọc. Còn 6/59 dòng ở mức med/low.
- Ảnh query là chữ viết tay lỗi hành thư còn corpus là khai thư in — mọi con số đo trên đúng một cặp phân phối này, chưa nói được gì về các lỗi viết khác.
- Recall của ANN chỉ đo ở $N = 26.044$; recall của HNSW thường giảm khi N tăng, nên 99,1% không được suy diễn cho mốc một triệu.

8. Kết luận và hướng tiếp theo

1. **Chuyển sang Chinese-CLIP.** Hơn baseline ResNet18 2,31 lần ở radical@k với chi phí gần như bằng không. Không scale tiếp lên ViT-H: đường cong đã quay đầu.
2. **Fine-tune là đòn bẩy lớn nhất, và đã đo được.** hit@1 trên scan thật đi từ 0,0847 lên **0,5932**, Phần 2 từ 0,4746 lên **0,8136**, với bằng chứng rằng phần cải thiện không đến từ ghi nhớ.
3. **Giữ tìm kiếm chính xác IndexFlatIP** cho tới khi corpus vượt khoảng 300.000 ký tự.
4. **Việc tiếp theo, theo thứ tự chi phí.** (a) *Hard negative mining*: hàm mất mát hiện rơi về 0,0068 chỉ sau 400 bước vì 24 lớp lấy ngẫu nhiên từ 23.440 thì phân biệt quá dễ — phải nhồi vào cùng batch những ký tự trông giống nhau thì mới còn tín hiệu để học. (b) Scan thêm ảnh thật để gỡ nút thắt $n = 59$. (c) Bổ sung dữ liệu thư pháp bút lông thật (MCCD, NomNaOCR) — thứ mà render font về nguyên tắc không sinh ra được.

Phụ lục A. Tái lập toàn bộ số liệu

Mọi con số trong báo cáo sinh ra từ các lệnh dưới đây, chạy tuần tự **từ trong thư mục dự án**. Mọi script đều phân giải ./images, ./output, ./test_images và các file .xlsx theo thư mục làm việc hiện tại, nên chạy từ chỗ khác sẽ hỏng.

Chuẩn bị môi trường. Cần Python 3.10+ và một GPU có ít nhất 6 GB trống.

```
# nếu máy đã có sẵn torch hệ thống, thêm --system-site-packages
python -m venv .venv
.venv/bin/pip install -r requirements.txt

# đặt hai file .xlsx vào thư mục gốc, giải nén images.zip -> ./images
bash download_fonts.sh # tải 7 font Hán-Nôm vào ./fonts
```

Mục 5.1 — so sánh backend. Mỗi lệnh cache embedding, chạy lại sẽ dùng cache.

```
.venv/bin/python search_all_chars_in_corpus.py --backend resnet18 --device cuda
.venv/bin/python search_all_chars_in_corpus.py --backend resnet18-gray --device cuda
.venv/bin/python search_all_chars_in_corpus.py --backend chinese-clip --device cuda
.venv/bin/python search_all_chars_in_corpus.py --backend chinese-clip-large \
--device cuda --dtype fp16
.venv/bin/python search_all_chars_in_corpus.py --backend chinese-clip-huge \
--device cuda --dtype fp16
.venv/bin/python search_all_chars_in_corpus.py --backend dinov2 --device cuda
.venv/bin/python benchmark_extractors.py # -> benchmark_summary.xlsx (Bảng 4)
```

Mục 5.2 và 5.3 — trần của chỉ số proxy, và so sánh tìm kiếm chính xác với ANN.

```
.venv/bin/python analyze_metric_ceiling.py # trần lý thuyết + oracle IDS
.venv/bin/python benchmark_search.py # -> output/benchmark_search.xlsx
```

Mục 3.3 — sinh dữ liệu huấn luyện. Bước này mất khoảng 20 phút trên 16 nhân CPU.

```
.venv/bin/python render_fonts.py # 131.604 ảnh -> output/rendered/
.venv/bin/python scan_augment.py --calibrate # Bảng 2
.venv/bin/python scan_augment.py --preview # bảng ảnh đề'nhìn bằng mắt

# KHÔNG cần --augment cho việc huấn luyện: finetune_glyph.py suy giảm ảnh trực tuyến.
# Cờ này chỉ dùng khi muốn xem trước tập suy giảm trên đĩa.
```

Mục 3.4 — fine-tune. Khoảng 30 phút, 3,7 GB VRAM.

```
.venv/bin/python finetune_glyph.py --smoke # 200 lớp, kiểm đường ống trước
.venv/bin/python finetune_glyph.py --epochs 6 --device cuda
# -> output/finetune/best.pt
```

Mục 4, 5.4 — đánh giá trên scan thật. Bỏ --backend chinese-clip-ft để lấy lại các con số zero-shot ở cùng bảng.

```
.venv/bin/python search_all_chars_in_corpus.py --backend chinese-clip-ft \
--dtype fp16 --device cuda

.venv/bin/python evaluate_test_images.py --backend chinese-clip-ft --dtype fp16 \
--device cuda --labels output/label_sheets/label_template.csv

# phép thử' độ nhạy: chỉ chấm trên 53 nhãn confidence=high
.venv/bin/python evaluate_test_images.py --backend chinese-clip-ft --dtype fp16 \
--device cuda --labels output/label_sheets/label_template.csv \
--part 2 --min-confidence high

# sinh lại phiếu gán nhãn (query + TOÀN BỘ ứng viên, kèm mã Unicode)
.venv/bin/python evaluate_test_images.py --sheets
```

Nhật ký huấn luyện đầy đủ, gồm cả lần ArcFace sụp đổ ở mục 6.3, nằm trong output/finetune/train.log. BENCHMARK.md giữ bản dài của mọi bảng phụ mà báo cáo này không chép lại.

Phụ lục B. Nội dung gói nộp

Bảng 9. Cấu trúc gói nộp.

Đường dẫn	Nội dung
src/	Toàn bộ mã nguồn do nhóm viết (xem bảng dưới).
data/test_images/	59 ảnh scan thật — tập đánh giá.
data/label_template.csv	Nhãn do người gán , kèm cột confidence. Không sinh lại được bằng cách chạy code.
data/DATA.md	Nguồn của từng tập dữ liệu và cách lấy phần không kèm theo.
model/	Trọng số fine-tune, hoặc liên kết tải nếu vượt hạn mức nộp.
report/	Báo cáo .docx và .pdf, cùng script sinh ra chúng.
BENCHMARK.md	Bản dài: mọi bảng, mọi lần chạy, mọi kết quả âm.
README.md	Hướng dẫn chạy nhanh.

Bảng 10. Mã nguồn do nhóm viết.

Script	Vai trò
feature_extractors.py	Sáu backend trích đặc trưng sau một giao diện chung.
search_all_chars_in_corpus.py	Phần 1: nhúng toàn corpus, cache, tìm top-k bằng Faiss.
search_use_QuocNgu_mapping.py	Phần 2: xếp hạng trong nhóm cùng âm, histogram và embedding.
benchmark_extractors.py	So sánh backend theo ba chỉ số proxy (Bảng 4).
analyze_metric_ceiling.py	Trần lý thuyết và trần oracle của radical@k (mục 5.2).
benchmark_search.py	So tìm kiếm chính xác với ANN (mục 5.3).
evaluate_test_images.py	Đánh giá trên scan thật, sinh phiếu gán nhãn (mục 4, 5.4).
render_fonts.py	Sinh cặp dương bằng render đa font (mục 3.3).
scan_augment.py	Mô hình suy giảm ảnh, đã hiệu chỉnh theo số đo (Bảng 2).
finetune_glyph.py	Fine-tune SupCon (mục 3.4), kèm nhánh ArcFace để tái lập mục 6.3.
build_report.py	Sinh chính báo cáo này.

Tài liệu và tài nguyên ngoài

Các mô hình pretrain **không** kèm trong gói nộp; chúng được tải tự động từ Hugging Face khi chạy lần đầu:

- Chinese-CLIP: `OFA-Sys/chinese-clip-vit-base-patch16`, `...-large-patch14`, `...-huge-patch14`
- DINOv2: `facebook/dinov2-base`
- ResNet18: trọng số ImageNet của torchvision

Font Hán-Nôm (BabelStone Han, HanaMin A/B, I.Ming, LXGW WenKai TC, Noto Sans/Serif TC, Zhuque Fangsong) tải bằng `download_fonts.sh`; tất cả đều là font tự do, nguồn ghi trong chính script đó.